

Documentation Technique Définitive & Manuel Opérationnel Assistant Médical IA Multimodal MedStral

Yassine Ouali

8 décembre 2025



FIGURE 1 – Your caption here

Résumé

Ce rapport consolide le cycle de vie logiciel complet, des fondations théoriques de l'apprentissage automatique dérivées des notebooks de recherche jusqu'à l'architecture microservice pratique déployée dans la couche applicative. Il sert de référence technique pour les ingénieurs ML, les équipes DevOps et les auditeurs de conformité.

Table des matières

1	Résumé Technique Exécutif	3
2	Architecture du Système & Gestion du Cycle de Vie	3
2.1	Cadre Central : La Colonne Vertébrale Asynchrone	3
2.2	Le Cycle de Requête en « Sandwich de Traduction »	3
2.3	Schéma de Persistance des Données & Sécurité	4
2.3.1	Stockage Transactionnel (SQLite)	4
2.3.2	Stockage Vectoriel (Pinecone)	4
2.4	Confidentialité & Gestion Éphémère des Fichiers	5
3	Documentation des Modèles d'Apprentissage Automatique	6
3.1	Le Moteur Cognitif : Mistral-7B Médical	6
3.2	Le Moteur Visuel : DenseNet CheXpert	6
3.3	Le Moteur Acoustique : Biomarqueurs VoiceSick	7
4	Manuel de Déploiement & Opérationnel	9
4.1	Séquence de Démarrage à Froid	9
4.2	Recommandations Matérielles	9
4.3	Limitations Connues	9

1 Résumé Technique Exécutif

L'**Assistant Médical IA Multimodal** est un microservice monolithique et asynchrone conçu pour démocratiser l'accès aux diagnostics médicaux de niveau triage. Contrairement aux systèmes unimodaux traditionnels (qui ne traitent que du texte *ou* des images), ce système met en œuvre une approche de **Fusion de Capteurs** (*Sensor Fusion*). Il agrège des flux de données disparates — descriptions textuelles cliniques, imagerie radiographique et signaux audio bio-acoustiques — en un pipeline de diagnostic unifié.

Le système est architecturé comme un moteur de **Génération Augmentée par Récupération (RAG)**. Cela signifie que les capacités génératives de son Grand Modèle de Langage (LLM) sont strictement limitées par un contexte récupéré de littérature médicale vérifiée, minimisant le risque d'« hallucinations » (sorties confiantes mais fausses). La couche applicative enveloppe ces modèles d'IA dans un framework **FastAPI** sécurisé et à haute concurrence, assurant la conformité aux normes de confidentialité des données via des pratiques de stockage éphémères.

2 Architecture du Système & Gestion du Cycle de Vie

Cette section détaille le « conteneur » qui héberge les modèles d'IA, gérant le flux de données de l'utilisateur final vers les moteurs d'inférence et vice-versa.

2.1 Cadre Central : La Colonne Vertébrale Asynchrone

L'application est construite sur **FastAPI**, un framework web moderne choisi spécifiquement pour son support natif de la bibliothèque `asyncio` de Python.

- **Pourquoi FastAPI ?** L'inférence d'apprentissage automatique est souvent « bloquante » (lourde en calcul), tandis que les opérations de base de données et de réseau sont « liées aux E/S » (*I/O bound*). FastAPI permet au serveur de traiter les requêtes HTTP entrantes de manière asynchrone tout en déchargeant les tâches d'inférence lourdes vers des pools de threads. Cela garantit qu'une analyse de radiographie de longue durée ne fige pas le serveur pour les autres utilisateurs essayant de se connecter ou de discuter.
- **Implémentation du Serveur :** L'application tourne sur **Uvicorn**, un serveur ASGI (*Asynchronous Server Gateway Interface*). Pour assurer la compatibilité au sein des environnements de développement comme Jupyter ou Google Colab, le système applique le correctif `nest_asyncio`, qui permet la réentrée dans la boucle d'événements — une solution de contournement critique pour exécuter des serveurs web asynchrones dans des notebooks.

2.2 Le Cycle de Requête en « Sandwich de Traduction »

Pour servir une base de patients mondiale, le système implémente un modèle de middleware unique appelé « Sandwich de Traduction ». Cela abstrait les barrières linguistiques des modèles médicaux centraux, qui sont entraînés principalement sur des données médicales en anglais.

1. Ingress (Le Pain Supérieur) :

- L'utilisateur soumet une requête dans sa langue maternelle (ex : Arabe : « »).

- Le **Middleware d'Authentification** intercepte cette requête en premier. Il valide le **JWT (JSON Web Token)** dans l'en-tête. Si la signature est invalide ou si le jeton a expiré ($TTL > 30$ min), la requête est rejetée immédiatement avec une erreur **401 Unauthorized**.
 - Une fois authentifié, le service **GoogleTranslator** détecte la langue source et traduit la charge utile en anglais (« I feel shortness of breath »).
2. **Traitement Central (La Viande) :**
- La requête en anglais est transmise au **Moteur RAG** (pour récupérer le contexte médical) puis au **LLM Mistral** (pour générer des conseils).
 - Le système invite le LLM avec la structure suivante :
« Basé sur le contexte suivant [Données Récupérées], répondez à la question de l'utilisateur : [Requête en Anglais] »
3. **Egress (Le Pain Inférieur) :**
- Le LLM génère une réponse en anglais (ex : « This could be asthma... »).
 - Le service de traduction convertit cette réponse anglaise *vers* la langue d'origine de l'utilisateur (Arabe).
 - La réponse JSON finale envoyée au client contient le texte localisé, créant une expérience utilisateur transparente où l'IA semble parler couramment la langue maternelle de l'utilisateur.

2.3 Schéma de Persistance des Données & Sécurité

L'intégrité des données est gérée par une approche de base de données hybride, séparant les données transactionnelles structurées des données vectorielles de haute dimension.

2.3.1 Stockage Transactionnel (SQLite)

- **Table Utilisateurs :** Stocke les identifiants. Le mot de passe n'est **jamais** stocké en texte clair. Le système utilise **Argon2id**, un algorithme de hachage « memory-hard » (difficile en mémoire).
- **Détail Technique :** Contrairement aux anciens algorithmes comme MD5 ou SHA256, Argon2id est conçu pour nécessiter une RAM significative pour calculer un hachage. Cela le rend résistant aux attaques par force brute basées sur GPU, car les attaquants ne peuvent pas paralléliser le processus de craquage efficacement sur des cartes graphiques.
- **Table Historique du Chat :** Enregistre la conversation à des fins d'audit. Cette table capture `user_id`, `timestamp`, `raw_query`, et `bot_response`. Ce journal est essentiel pour le futur **Apprentissage par Renforcement à partir de Rétroaction Humaine (RLHF)** afin d'affiner davantage le modèle.

2.3.2 Stockage Vectoriel (Pinecone)

Le système utilise un index Pinecone *serverless* (région `us-east-1`) pour stocker les plongements (embeddings) de manuels et documents médicaux.

- **Métrique de Recherche :** Le système utilise la **Similarité Cosinus** plutôt que la Distance Euclidienne. Dans les espaces de haute dimension (384 dimensions), l'*angle* entre les vecteurs est une mesure plus fiable de la similarité sémantique que la magnitude ou la distance.

2.4 Confidentialité & Gestion Éphémère des Fichiers

Le traitement des images médicales (Rayons X) et des enregistrements audio implique des contraintes de confidentialité strictes (conformité RGPD/HIPAA).

- **Le Problème** : De nombreuses bibliothèques d'apprentissage automatique Python (`cv2.imread`, `librosa.load`) sont des *wrappers* autour de backends C/C++ qui nécessitent un chemin de fichier physique sur le disque ; elles ne peuvent pas facilement lire des octets de fichier directement depuis la RAM.
- **La Solution** : Le système utilise la classe `tempfile.NamedTemporaryFile` de Python.
 1. Lorsqu'un fichier est téléchargé, il est diffusé (*streamed*) vers un emplacement temporaire dans le répertoire `/tmp` du serveur.
 2. Le chemin du fichier est passé au moteur d'inférence.
 3. De manière cruciale, un bloc `try...finally` garantit que la commande `os.unlink(path)` est appelée immédiatement après l'inférence. Cela garantit que les données du patient n'existent sur le disque du serveur que pendant les quelques millisecondes requises pour le traitement, prévenant toute fuite de données.

3 Documentation des Modèles d'Apprentissage Automatique

Cette section déconstruit les méthodologies d'entraînement, les choix architecturaux et les configurations d'hyperparamètres pour chacune des trois modalités d'IA.

3.1 Le Moteur Cognitif : Mistral-7B Médical

Artefact Source : `mistral-finetuning.ipynb`

Le « cerveau » du système est un Grand Modèle de Langage (LLM) personnalisé. Nous n'avons pas simplement utilisé un modèle de base avec des « prompts » ; nous avons altéré ses poids via un ajustement fin (*fine-tuning*) pour l'aligner avec les normes de sécurité médicale.

- **Sélection du Modèle de Base : Mistral-7B-Instruct-v0.2.** Ce modèle a été choisi pour son ratio élevé de capacité de raisonnement par paramètre. Il surpasse Llama-2-13B sur de nombreux benchmarks tout en étant suffisamment petit pour tenir sur un seul GPU.
- **Quantification (Le « Rayon Rétrécissant ») :**
 - L'exécution d'un modèle de 7 milliards de paramètres nécessite généralement ~28 Go de VRAM (en précision FP32). Nous employons une quantification **4-bit Normal Float (NF4)** via la bibliothèque `bitsandbytes`.
 - **Détail Technique :** NF4 est un type de données optimisé pour la distribution normale des poids de réseaux neuronaux. En mappant les poids à cette précision inférieure, nous réduisons l'utilisation de la VRAM à ~5 Go, rendant le système déployable sur du matériel abordable (comme les NVIDIA T4).
 - **Double Quantification :** Nous quantifions également les constantes de quantification elles-mêmes, extrayant une efficacité supplémentaire.
- **Méthodologie d'Ajustement Fin (QLoRA) :**
 - Nous utilisons l'**Adaptation de Rang Faible (LoRA)**. Au lieu de réentraîner les 7 milliards de paramètres (ce qui est prohibitif en calcul), nous gelons le modèle de base et attachons de petites matrices « adaptateurs » entraînaibles aux couches d'Attention (`q_proj`, `v_proj`).
 - **Rang ($r = 16$) :** Le rang détermine la complexité de l'adaptateur. Un rang de 16 permet au modèle d'apprendre des nuances médicales spécifiques sans surapprentissage ni oubli catastrophique de ses connaissances générales en anglais.
 - **Modules Cibles :** Les adaptateurs sont attachés à toutes les couches linéaires (`k_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`) pour maximiser la surface d'apprentissage.
- **Alignement de Sécurité :** Les données d'entraînement incluaient des exemples de « refus » — des scénarios où le modèle *ne doit pas* répondre (ex : « Prescrivez-moi de l'Oxycodone »). Le modèle apprend à déférer aux professionnels humains dans ces cas.

3.2 Le Moteur Visuel : DenseNet CheXpert

Artefact Source : `vision.ipynb`

Les « yeux » du système sont propulsés par un Réseau Neuronal Convolutif (CNN) spécifiquement ingénieur pour l'analyse de texture radiologique.

- **Architecture du Modèle : DenseNet121**
 - *Pourquoi pas ResNet ?* Dans ResNet, les caractéristiques sont ajoutées (sommees). Dans DenseNet, les caractéristiques sont **concaténées**. Cela signifie que les caractéristiques apprises dans la toute première couche sont passées directement à la couche finale. Pour les rayons X, où une opacité minuscule et subtile (pneumonie) pourrait être perdue dans les couches profondes, cette « réutilisation des caractéristiques » est critique.
- **Stratégie d'Étiquetage « U-Ones » :**
 - Le jeu de données CheXpert contient des étiquettes incertaines (où les radiologues n'étaient pas sûrs).
 - **Politique :** Nous mappons toutes les étiquettes « Incertain » (-1) à « Positif » (1).
 - **Raisonnement :** Dans un outil de dépistage, les Faux Positifs (signaler une personne saine) sont acceptables, mais les Faux Négatifs (manquer une personne malade) sont dangereux. Cette politique force le modèle à être hyper-sensible.
- **Prétraitement des Entrées :**
 - Les images sont redimensionnées à **320x320 pixels**. C'est plus élevé que le standard 224x224 utilisé dans ImageNet, permettant au modèle de voir des détails plus fins dans le tissu pulmonaire.
 - La normalisation utilise les moyennes/écarts-types d'ImageNet, car le backbone a été pré-entraîné sur ImageNet.
- **IA Explicable (Grad-CAM) :**
 - Le système ne se contente pas de sortir « Pneumonie : 88% ». Il génère une **Carte de Chaleur**.
 - **Mécanisme :** Il calcule le gradient du score « Pneumonie » par rapport aux dernières cartes de caractéristiques convolutionnelles. Si le modèle regarde le poumon supérieur droit pour prendre sa décision, les gradients à cet endroit seront élevés. Ces gradients sont visualisés comme une superposition rouge, montrant au médecin où l'IA regarde.

3.3 Le Moteur Acoustique : Biomarqueurs VoiceSick

Artefact Source : voiceSick.ipynb

Les « oreilles » du système utilisent une « Architecture en Cascade » sophistiquée pour traiter les formes d'onde audio brutes de la toux et de la respiration.

- **Ingénierie des Caractéristiques (L'Empreinte Digitale) :**
 - L'audio brut n'a pas de sens pour un classifieur standard. Nous transformons l'audio en caractéristiques spectrales utilisant **Librosa**.
 - **MFCCs (Coefficients Cepstraux de Fréquence Mel) :** Capturent le « timbre » ou la texture du son.
 - **Contraste Spectral :** Détecte le caractère « boueux » vs « clair » du son (utile pour détecter du fluide dans les poumons).
 - **Agrégation Statistique :** Nous ne prenons pas simplement les caractéristiques; nous calculons la Moyenne, la Variance, l'Asymétrie (*Skewness*) et l'Aplatissement (*Kurtosis*) de chaque caractéristique dans le temps. Cela crée une « empreinte digitale » statistique robuste de la voix du patient.
- **Étape 1 : Le Dépisteur (XGBoost)**
 - C'est un modèle d'apprentissage automatique classique et rapide.

- **Objectif** : Classification Binaire (Sain vs Malade).
- **Optimisation** : Nous avons utilisé la **Statistique J de Youden** pour ajuster le seuil de décision. Le seuil par défaut est 0.5 (50% de probabilité). Nous l'avons déplacé à **0.2204**. Cela signifie que si le modèle est même *légèrement* soupçonneux (22% confiant), il signale le patient comme « Malade ». Cela s'aligne avec le principe médical de dépistage à haute sensibilité.
- **Étape 2 : Le Spécialiste (Hybride CNN-LSTM)**
 - Si le Dépisteur dit « Malade », ce modèle Deep Learning s'active.
 - **Architecture** :
 - **Couches CNN (Conv2D)** : Scannent le spectrogramme audio comme une image pour trouver des motifs locaux (ex : un sifflement).
 - **Couches LSTM (Long Short-Term Memory)** : Analysent la *séquence* du son dans le temps (ex : la toux s'estompe-t-elle ou se termine-t-elle brusquement ?).
 - **Tâche** : Diagnostic Différentiel. Il tente de distinguer entre COVID-19, Asthme et grippe générale basé sur l'évolution temporelle du son de la toux.

4 Manuel de Déploiement & Opérationnel

4.1 Séquence de Démarrage à Froid

Lorsque le serveur démarre, il initie une phase de chargement intensive en ressources :

1. **Allocation VRAM** : Le modèle Mistral réserve ~5 Go de VRAM. Si le GPU a moins de 6 Go, l'application échouera avec une erreur `CUDA Out Of Memory (OOM)`.
2. **Fusion d'Adaptateurs** : Les adaptateurs LoRA sont chargés depuis le disque et dynamiquement « fusionnés » avec le modèle de base en mémoire utilisant `PeftModel.from_pretrained()`.
3. **Connexion Index Vectoriel** : Le `rag_service` tente une poignée de main avec Pinecone. Si l'index n'existe pas, il déclenche une routine de création (bien que cela prenne des minutes, donc il est généralement pré-provisionné).

4.2 Recommandations Matérielles

- **Spécification Minimale** : NVIDIA T4 (16 Go VRAM) ou RTX 3060 (12 Go VRAM). 16 Go de RAM Système.
- **Stockage** : 20 Go d'espace libre (pour les Poids des Modèles + Conteneurs Docker).
- **Réseau** : Internet stable requis pour Pinecone (Base de données Vectorielle Cloud) et les appels API Google Translate.

4.3 Limitations Connues

- **Latence** : Le « Sandwich de Traduction » ajoute environ 1,5 seconde de latence par requête. La traduction en temps réel est coûteuse en calcul.
- **Concurrence** : L'implémentation actuelle traite les requêtes d'inférence sur le GPU principal. Bien que FastAPI gère les requêtes web de manière asynchrone, si 10 utilisateurs téléchargent des rayons X simultanément, ils seront traités séquentiellement (Premier Entré, Premier Sorti) sur le GPU. La mise à l'échelle nécessite un système de file d'attente (comme Celery/Redis) et plusieurs travailleurs GPU.

